UNIVERSIDADE FEDERAL DE ALAGOAS – UFAL
INSTITUTO DE COMPUTAÇÃO – IC
PROGRAMA DE PÓS-GRADUAÇÃO EM INFORMÁTICA – PPGI

Build a Large Language Model

(From Scratch)

Sebastian Raschka

Inteligência Artificial Generativa · Professor: Prof. Dr. Baldoino Fonseca

Equipe: Christiane Cabral · Elmo Araújo Filho · Lilian Neves · Rafael Jatobá · Victor Lemos

2026.2

Overview

Vamos explorar os principais conceitos por trás dos Large Language Models (LLMs), desde seus fundamentos até a construção de um modelo de linguagem do zero.

Tópicos de hoje

- Understanding large language models
- Working with text data
- Coding attention mechanisms
- Implementing a GPT model from scratch to generate text

Próximos tópicos

- Pretraining on unlabeled data
- Fine-tuning for classification
- Fine-tuning to follow instructions

Compreendendo os Grandes Modelos de Linguagem

Conceitos fundamentais dos LLMs, a arquitetura Transformer e um guia passo a passo para construir um LLM do zero em código.

Principais tópicos

- Explicações em alto nível dos conceitos fundamentais por trás dos modelos de linguagem de grande escala (LLMs);
- Insights sobre a arquitetura Transformer, a partir da qual os LLMs são derivados;
- Um plano para construir um LLM do zero.

Introdução aos LLMs

- Evolução do PLN: os LLMs representam uma evolução das técnicas tradicionais de Processamento de Linguagem Natural, ampliando a capacidade de trabalhar com linguagem humana.

- Maior capacidade linguística: enquanto modelos anteriores eram geralmente desenvolvidos para tarefas específicas, os LLMs podem realizar diversas tarefas de linguagem a partir de um mesmo modelo.

- Base tecnológica: essa evolução está relacionada principalmente à arquitetura Transformer e ao treinamento com grandes volumes de dados textuais.

O que é um LLM?

Um Grande Modelo de Linguagem (LLM) é uma rede neural profunda treinada com grandes volumes de textos para compreender e gerar linguagem natural. Seu tamanho está relacionado ao grande número de parâmetros e à quantidade de dados utilizados em seu treinamento.

IA Generativa: LLMs são uma forma de IA generativa (GenAI), capazes de criar textos, imagens e outras mídias.

Deep Learning: Redes neurais com múltiplas camadas que aprendem automaticamente a partir dos dados, sem extração manual de características.

Machine Learning: Algoritmos que aprendem regras a partir dos dados, permitindo previsões e decisões sem programação explícita.

Relação entre Inteligência Artificial, Machine Learning, Deep Learning, LLMs e IA Generativa

Como esses conceitos se relacionam e se organizam de forma hierárquica, mostrando que os LLMs estão inseridos no contexto do Deep Learning e podem integrar aplicações de IA Generativa.

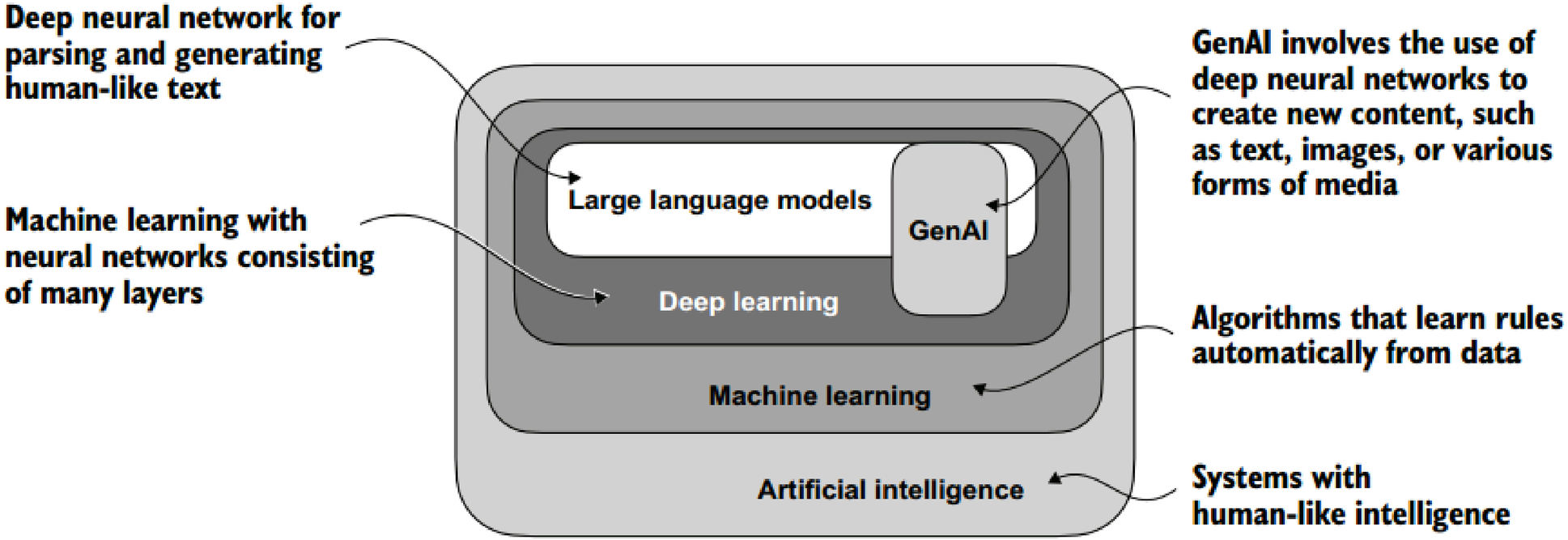


Figura 1.1 A representação hierárquica da relação entre os diferentes campos sugere que os LLMs representam uma aplicação específica de técnicas de Deep Learning, utilizando sua capacidade de processar e gerar textos semelhantes aos produzidos por humanos.
O Deep Learning é uma área especializada do Machine Learning que se concentra no uso de redes neurais com múltiplas camadas. O Machine Learning e o Deep Learning têm como objetivo implementar algoritmos que permitem aos computadores aprender com os dados e realizar tarefas que normalmente exigem inteligência humana.

Aplicações dos LLMs

Os LLMs possuem uma ampla variedade de aplicações em diversos domínios, automatizando praticamente qualquer tarefa que envolva análise e geração de texto.

Tradução e Geração de Texto: Tradução automática, geração de textos originais, resumo de documentos e análise de sentimentos.

Assistentes Virtuais: Chatbots sofisticados como ChatGPT e Gemini ampliam mecanismos de busca e respondem consultas complexas.

Recuperação de Conhecimento: Busca em documentos técnicos, sumarização de páginas extensas e respostas a questões em áreas como medicina e direito.

Comunicação entre usuários e sistemas de IA

A interface permite que o usuário interaja com o modelo por meio de linguagem natural, fornecendo instruções que orientam a geração de diferentes tipos de conteúdo.

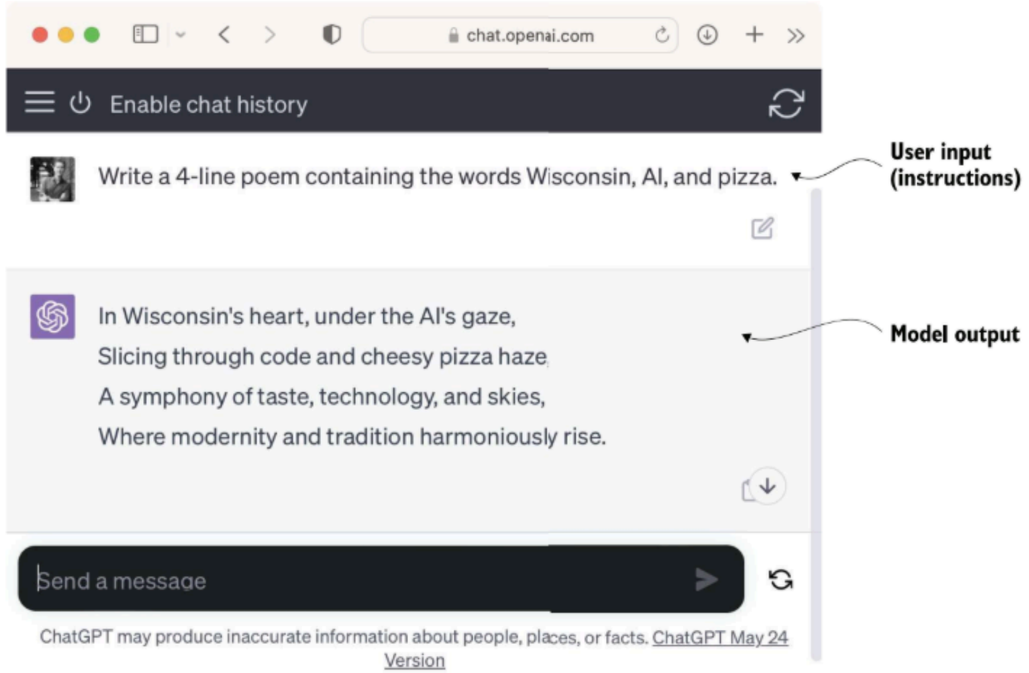


Figura 1.2 As interfaces de LLMs permitem a comunicação em linguagem natural entre usuários e sistemas de IA. Esta captura de tela mostra o ChatGPT escrevendo um poema de acordo com as especificações do usuário.

Etapas de Construção e Utilização de LLMs

As principais etapas envolvidas na construção e utilização de um LLM vão desde a preparação dos dados e o treinamento do modelo até sua aplicação na geração de respostas e realização de diferentes tarefas de linguagem natural.

Construção do LLM: Preparação e amostragem dos dados, mecanismo de atenção e implementação da arquitetura LLM para pré-treinamento.

Modelo Fundacional: Loop de treinamento, avaliação do modelo e carregamento de pesos pré-treinados publicamente disponíveis para criar um modelo-base.

Fine-Tuning: Ajuste do LLM pré-treinado para criar um classificador de textos (com dados rotulados) ou um assistente pessoal (com dados de instruções).

Pré-treinamento e ajuste de LLMs

O processo inicia com o treinamento em grandes volumes de textos para prever o próximo token e, posteriormente, o modelo pode ser ajustado com dados menores e específicos para determinada tarefa.

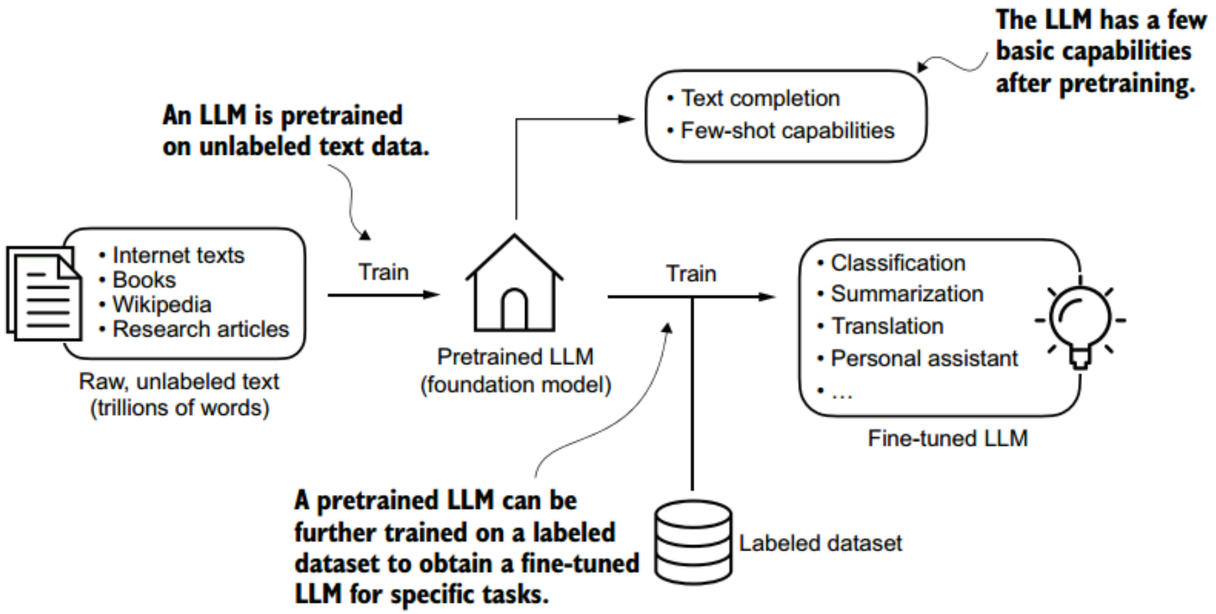


Figura 1.3 O pré-treinamento de um LLM envolve a previsão do próximo token em grandes conjuntos de dados textuais. Posteriormente, o modelo pré-treinado pode ser ajustado (fine-tuning) utilizando um conjunto de dados menor e rotulado.

Introdução à Arquitetura Transformer

A arquitetura Transformer, apresentada em 2017, transformou o desenvolvimento de modelos de linguagem ao permitir um processamento mais eficiente das relações contextuais entre os elementos do texto. Atualmente, está na base de muitos LLMs modernos.

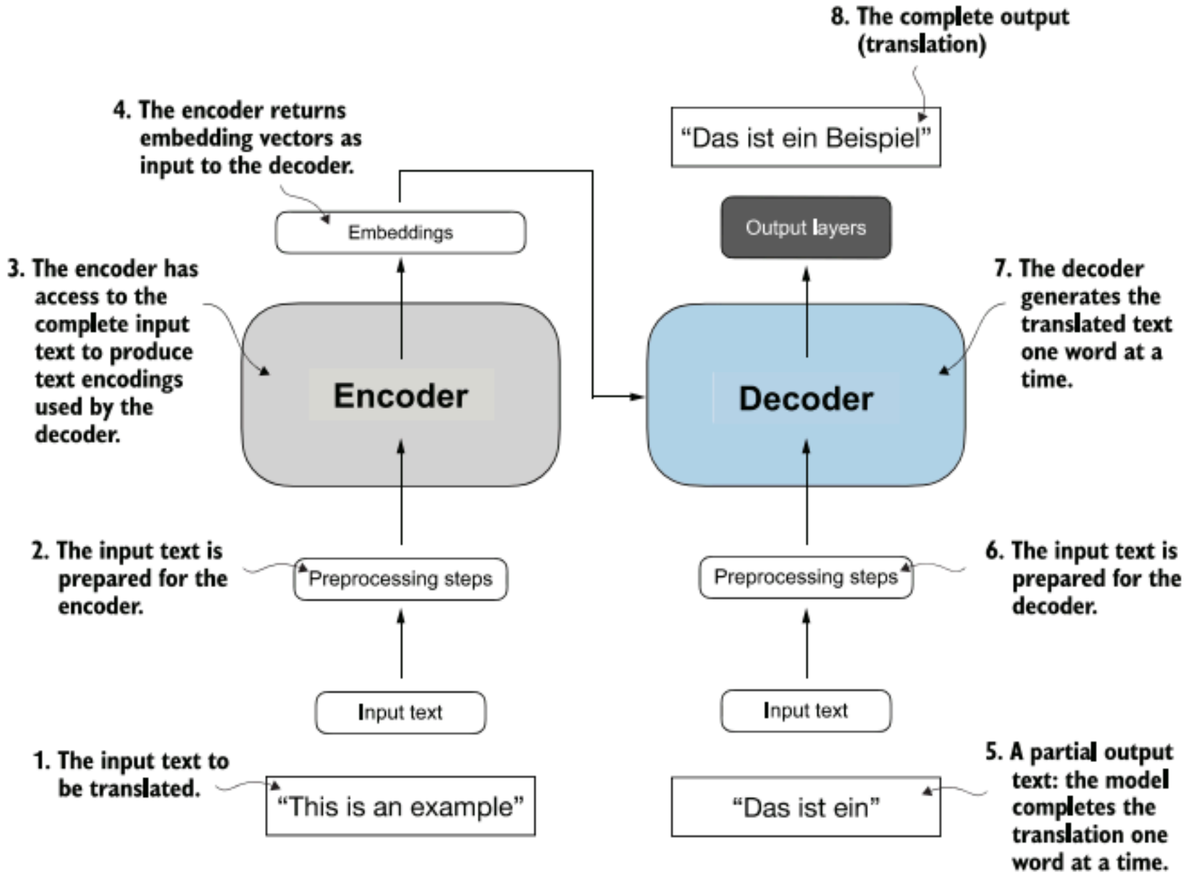
Arquitetura Encoder–Decoder: O encoder processa o texto de entrada e gera representações numéricas contextualizadas. O decoder utiliza essas representações para produzir o texto de saída.

Mecanismo de Self-Attention: Permite ao modelo identificar quais tokens são mais relevantes entre si, capturando relações contextuais e dependências ao longo da sequência.

BERT e GPT: A arquitetura deu origem a diferentes modelos, como BERT, baseado no encoder e voltado à compreensão do texto, e GPT, baseado no decoder e voltado à geração de texto.

Transformer × LLMs: Transformer é uma arquitetura de redes neurais, enquanto LLM é um modelo de linguagem treinado em grandes volumes de dados. Muitos LLMs atuais são baseados em Transformers, mas os conceitos não são sinônimos.

Arquitetura original do Transformer para tradução de idiomas

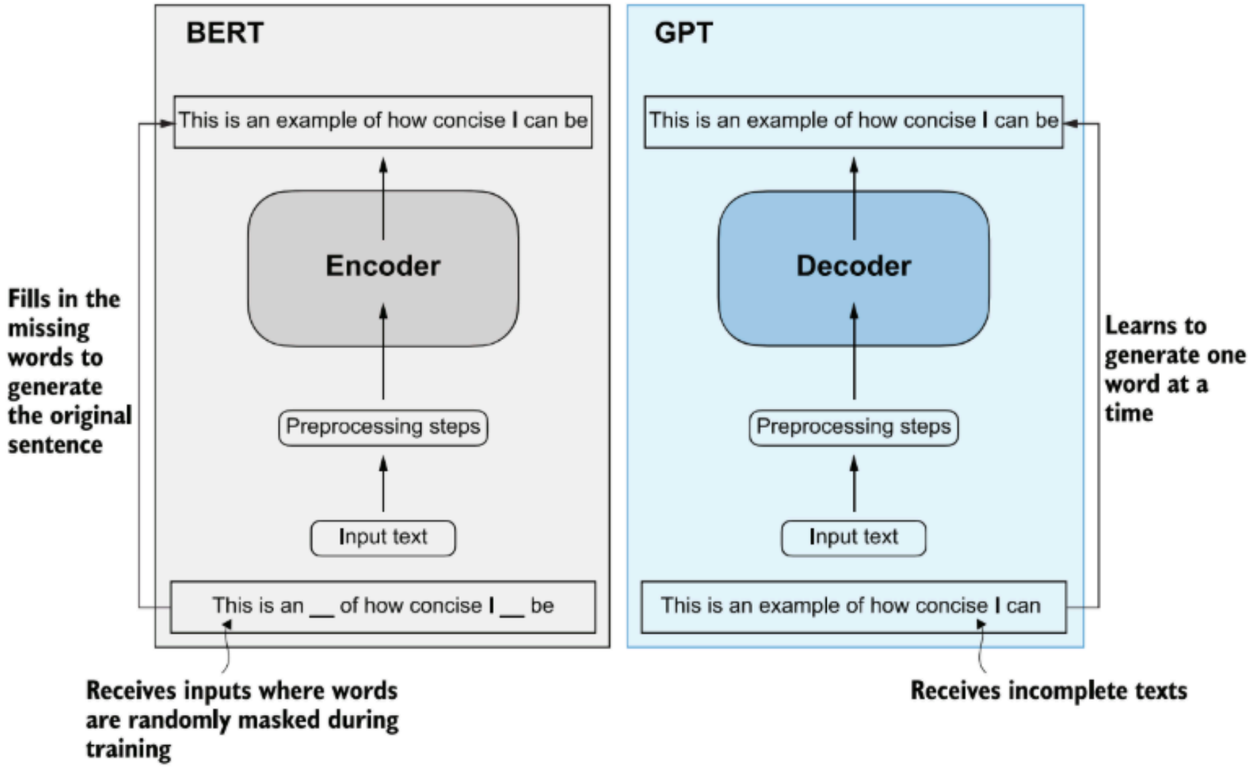


O Transformer é composto por duas partes:

(a) um codificador (encoder) que processa o texto de entrada e produz uma representação de embedding (uma representação numérica que captura muitos fatores diferentes do texto)
(b) um decodificador (decoder) que pode ser usado para gerar o texto traduzido, uma palavra de cada vez. Esta figura mostra a etapa final do processo de tradução, na qual o decodificador precisa gerar apenas a palavra final "Beispiel", completando a tradução da entrada original ("This is an example") para uma frase parcialmente traduzida ("Das ist ein").

Figura 1.4 Uma representação simplificada da arquitetura Transformer original, que é um modelo de aprendizagem profunda para tradução de idiomas.

Codificador e decodificador na arquitetura Transformer e sua relação com os LLMs

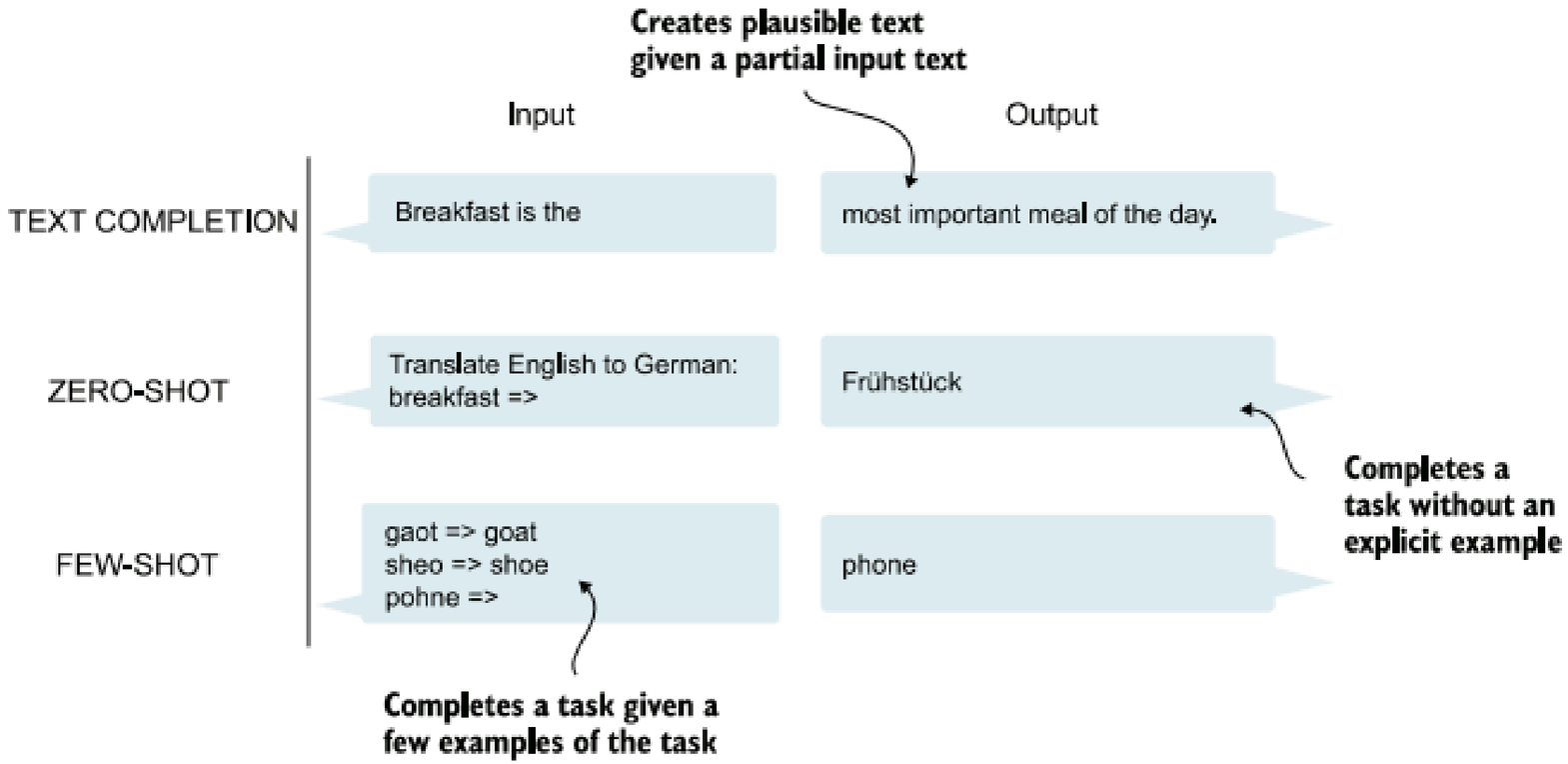


À esquerda, o segmento do codificador exemplifica os LLMs semelhantes ao BERT, que se concentram na previsão de palavras mascaradas e são utilizados principalmente para tarefas como classificação de textos.

À direita, o segmento do decodificador apresenta os LLMs semelhantes ao GPT, projetados para tarefas generativas e para produzir sequências de texto coerentes.

Figura 1.5 Uma representação visual dos submódulos de codificador (encoder) e decodificador (decoder) do Transformer.

Capacidades de LLMs semelhantes ao GPT em configurações few-shot e zero-shot



Às vezes, é útil fornecer exemplos da tarefa juntamente com a entrada, o que é conhecido como configuração de poucos exemplos (few-shot). No entanto, os LLMs semelhantes ao GPT também são capazes de realizar tarefas sem um exemplo específico, o que é denominado configuração de zero exemplos (zero-shot).

Figura 1.6 Além da conclusão de textos, os LLMs semelhantes ao GPT podem realizar diversas tarefas com base em suas entradas, sem a necessidade de novo treinamento, fine-tuning ou alterações específicas na arquitetura do modelo para cada tarefa.

Utilizando grandes conjuntos de dados

Os LLMs são treinados com grandes volumes de dados textuais, abrangendo diversos temas e linguagens. Esses dados, que podem conter bilhões de palavras, permitem aos modelos aprender padrões e relações da linguagem. O GPT-3 exemplifica a dimensão e diversidade desses conjuntos de dados.

Table 1.1 The pretraining dataset of the popular GPT-3 LLM

Dataset name	Dataset description	Number of tokens	Proportion in training data
CommonCrawl (filtered)	Web crawl data	410 billion	60%
WebText2	Web crawl data	19 billion	22%
Books1	Internet-based book corpus	12 billion	8%
Books2	Internet-based book corpus	55 billion	8%
Wikipedia	High-quality text	3 billion	3%

A Tabela 1.1 apresenta a quantidade de tokens, em que um token é uma unidade de texto que o modelo processa. A quantidade de tokens em um conjunto de dados é aproximadamente equivalente ao número de palavras e caracteres de pontuação presentes no texto.

Um olhar mais aprofundado sobre a arquitetura do GPT

O GPT é um modelo de linguagem que evoluiu com o aumento de parâmetros e dados de treinamento, permitindo tarefas como tradução e classificação.
Seu ajuste fino com instruções contribuiu para o desenvolvimento do ChatGPT.



Figura 1.7 Na tarefa de pré-treinamento de predição da próxima palavra dos modelos GPT, o sistema aprende a prever a palavra seguinte em uma frase observando as palavras que aparecem antes dela. Essa abordagem ajuda o modelo a compreender como palavras e frases normalmente se relacionam na linguagem, formando uma base que pode ser aplicada a diversas outras tarefas.

Arquitetura do GPT e processamento unidirecional

**As arquiteturas baseadas em GPT ampliaram a escala do Transformer original.
Modelos mais recentes, como o Llama, seus princípios fundamentais permanecem semelhantes, tornando essencial compreender a arquitetura do GPT.**

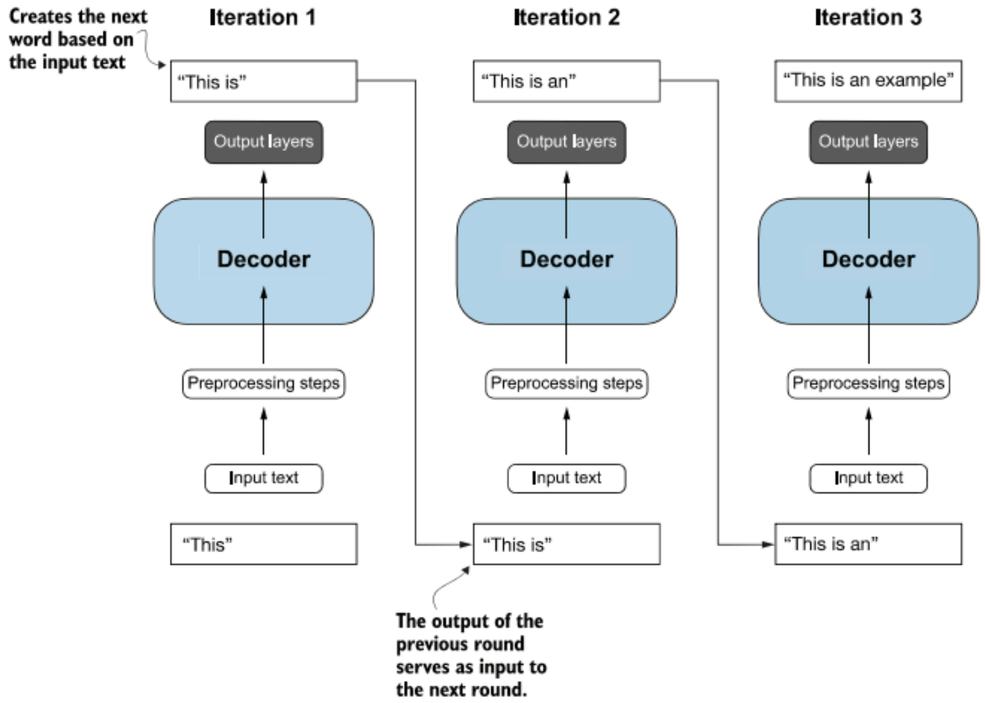


Figura 1.8 A arquitetura do GPT utiliza apenas a parte decodificadora do Transformer original. Ela foi projetada para o processamento unidirecional, da esquerda para a direita, sendo adequada para tarefas de geração de texto e previsão da próxima palavra, produzindo o texto de forma iterativa, uma palavra de cada vez.

Construindo um modelo de linguagem de grande escala

Construção de um modelo de linguagem do zero, utilizando a arquitetura GPT como referência e desenvolvendo o modelo em três etapas principais.

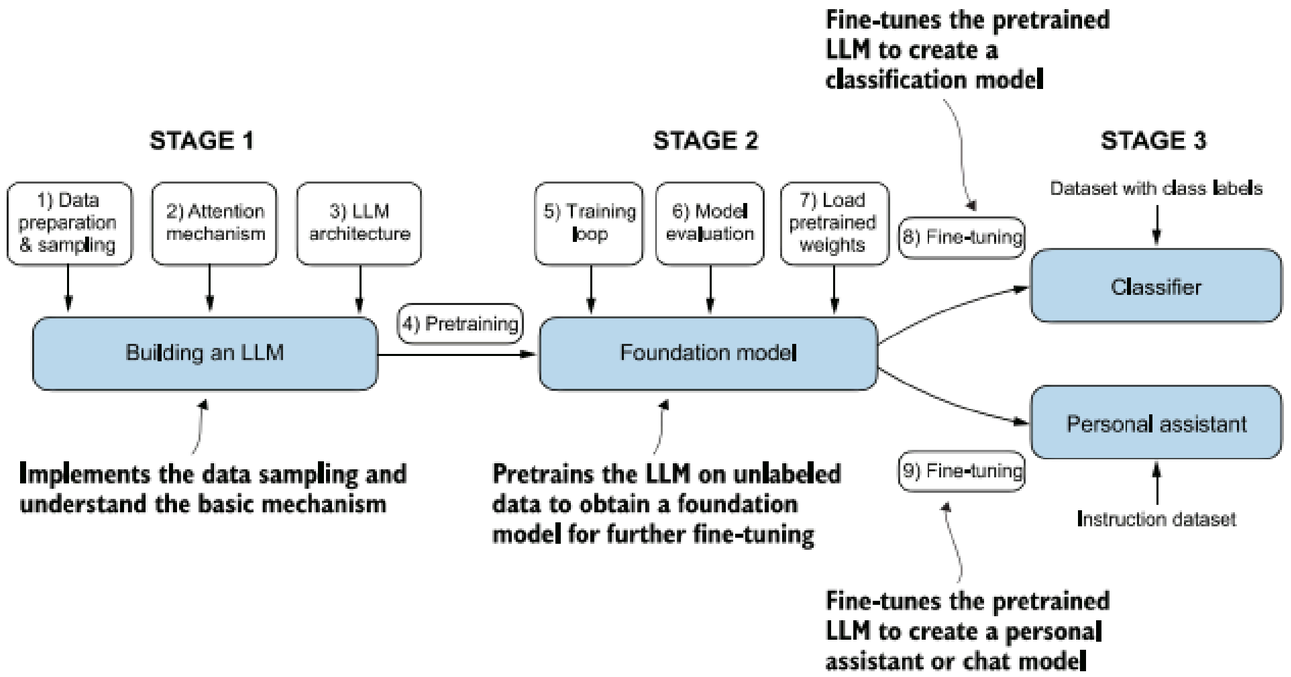


Figura 1.9 As três principais etapas para programar um LLM são: implementar a arquitetura do LLM e o processo de preparação dos dados (etapa 1); realizar o pré-treinamento do LLM para criar um modelo de base (etapa 2); e ajustar o modelo de base para transformá-lo em um assistente pessoal ou classificador de texto (etapa 3).

Capítulo 2

Trabalhando com dados de texto

Como transformar texto bruto em uma entrada numérica para um LLM: tokenização, vocabulário, embeddings e amostragem de dados.

Principais tópicos

- Preparação de textos para o treinamento de modelos de linguagem de grande escala (LLMs);
- Divisão do texto em tokens de palavras e subpalavras;
- Codificação por pares de bytes (Byte Pair Encoding – BPE) como uma forma mais avançada de tokenizar textos;
- Amostragem de exemplos de treinamento utilizando uma abordagem de janela deslizante;
- Conversão de tokens em vetores que alimentam um modelo de linguagem de grande escala (LLM).

Preparando o texto para o LLM

Antes do treinamento, o texto bruto passa por etapas de pré-processamento: Tokenization, conversão em Token IDs e criação de Embeddings.

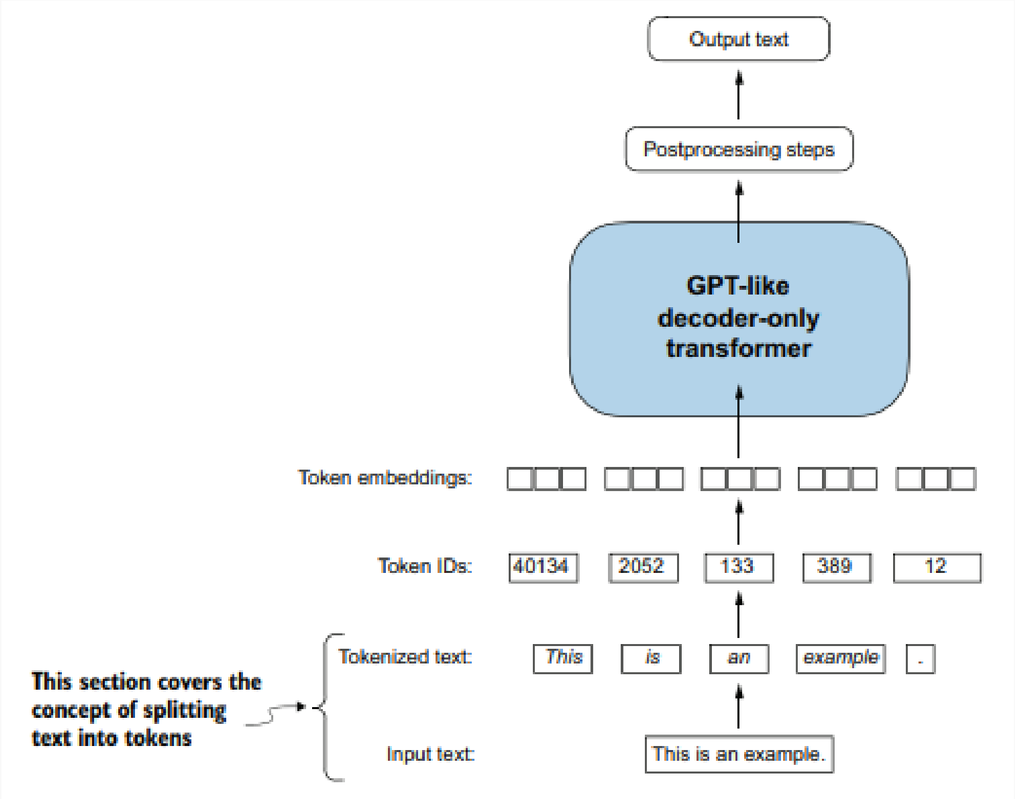


Figura 2.4. Visão geral do processamento de texto em um LLM. O texto de entrada é dividido em tokens (palavras ou caracteres especiais), convertido em token IDs e transformado em embeddings antes de chegar ao Transformer.

Embeddings: convertendo dados em vetores

Um modelo de embedding transforma cada dado em um vetor numérico denso.

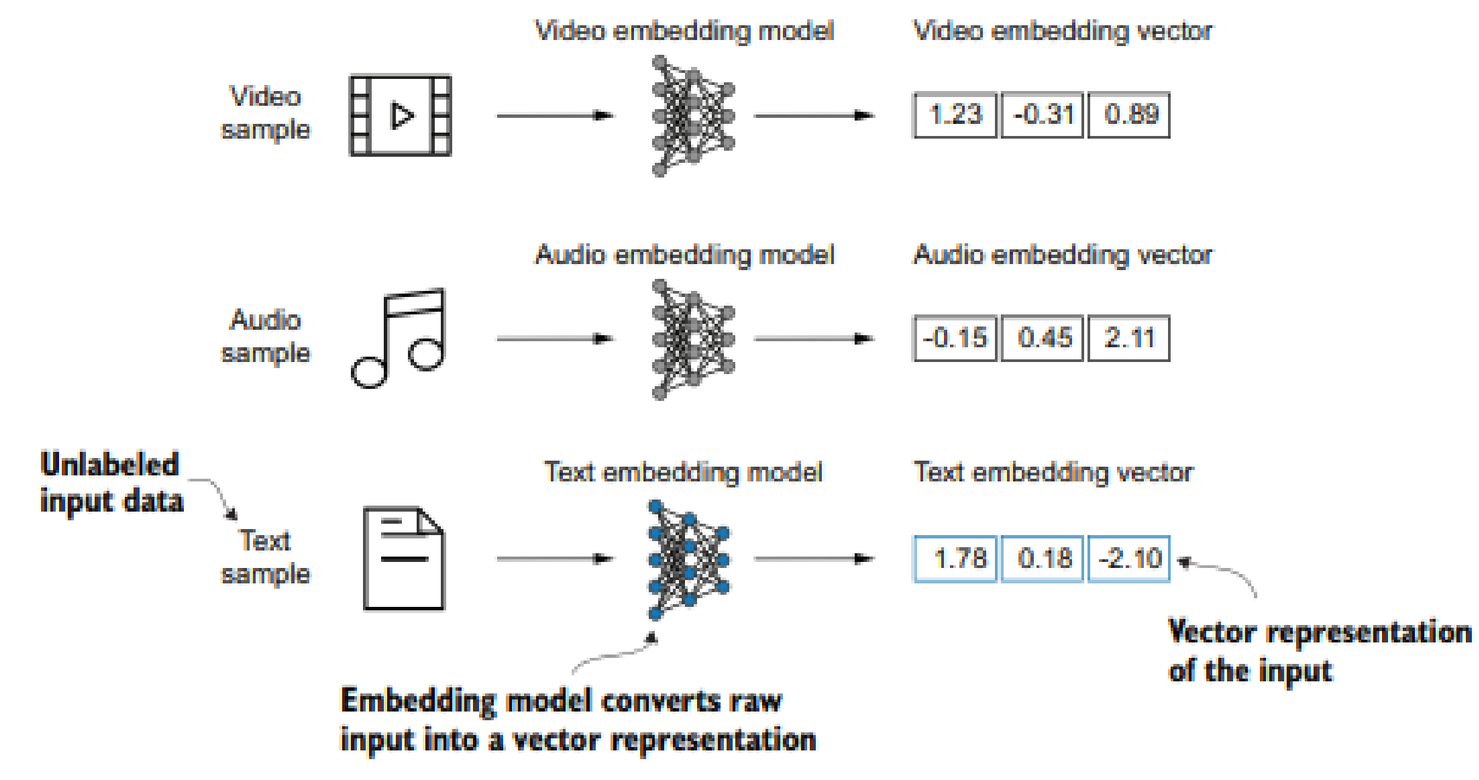


Figura 2.2. Cada tipo de dado exige seu próprio modelo de embedding. Na ilustração, amostras de vídeo, áudio e texto são convertidas em vetores numéricos de três dimensões.

Word Embeddings (Embeddings de Palavras)

Palavras com significados parecidos ficam próximas no espaço vetorial. Nos LLMs, os embeddings são aprendidos durante o treinamento e podem ter milhares de dimensões (12.288 no GPT-3).

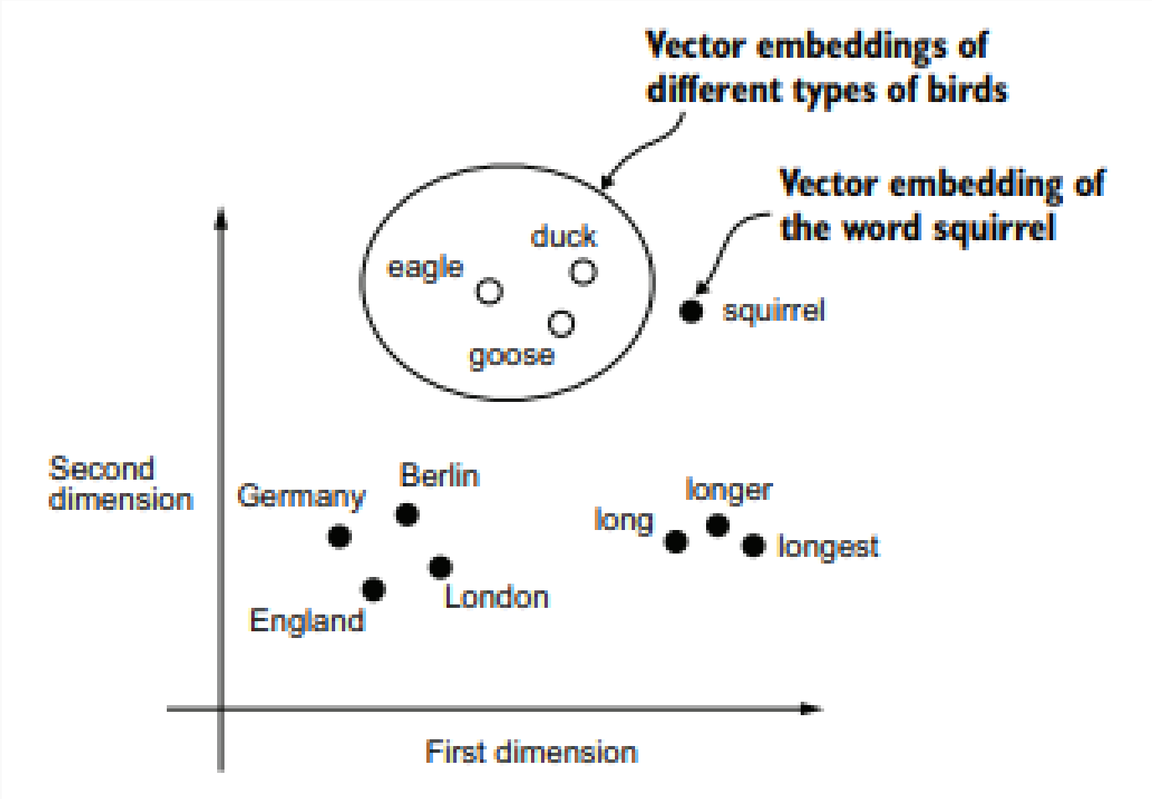


Figura 2.3. Embeddings de palavras representados em duas dimensões. Palavras relacionadas, como tipos de aves, aparecem agrupadas e distantes de outros conceitos, como países e cidades.

Tokenization (Tokenização)

Tokenization é o processo de dividir o texto em unidades menores, os tokens, que podem ser palavras ou caracteres especiais, como a pontuação.

Divisão do texto

Com expressões regulares, o texto é separado nos espaços e na pontuação. A frase “Hello, world. Is this-- a test?” gera 10 tokens.

Maiúsculas e minúsculas

O texto não é convertido para minúsculas: a capitalização ajuda o modelo a distinguir nomes próprios de substantivos comuns e a entender a estrutura das frases.

Espaços em branco

Removê-los reduz o uso de memória e de processamento, mas mantê-los pode ser importante em dados sensíveis à estrutura, como código Python.

Vocabulary (Vocabulário) e Token IDs

O vocabulário associa cada token único do conjunto de treinamento a um número inteiro, o token ID. No conto “The Verdict”, usado no livro, são 1.130 tokens únicos.

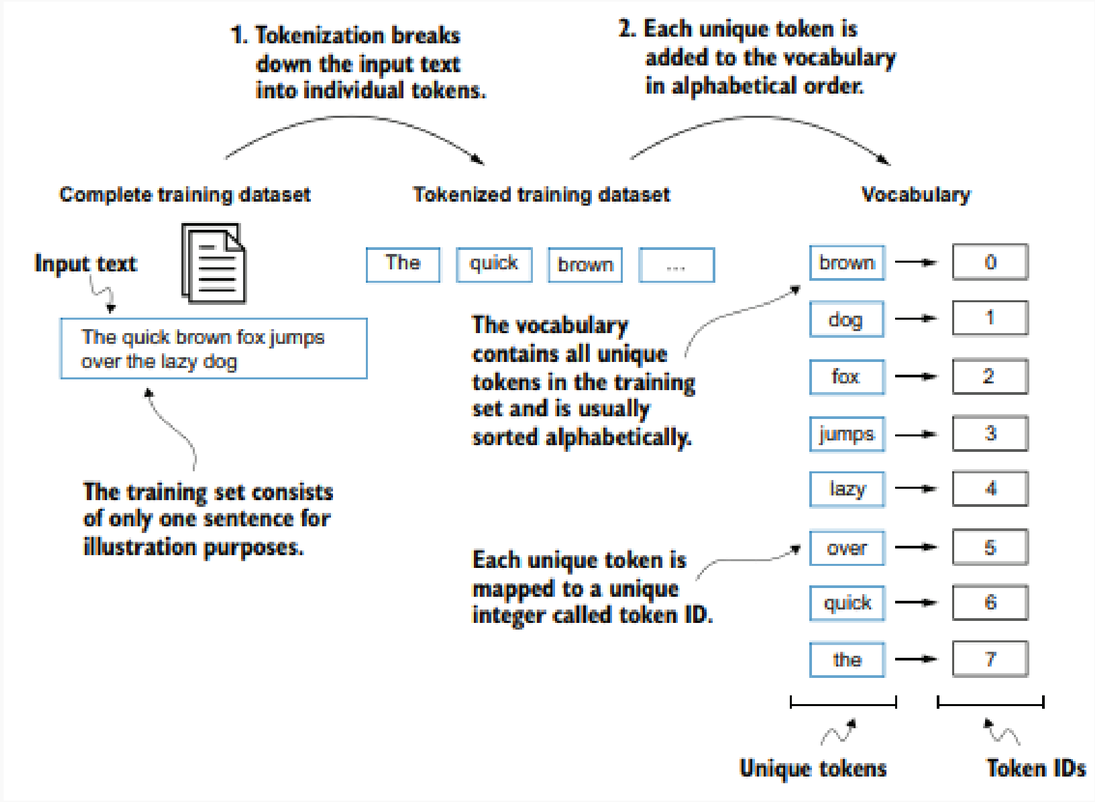


Figura 2.6. Construção do vocabulário: o texto de treinamento é dividido em tokens, os repetidos são removidos e os tokens únicos são ordenados alfabeticamente e numerados. O exemplo é propositalmente pequeno e não inclui pontuação.

Tokenizer: encode e decode

Um tokenizador implementa dois métodos:

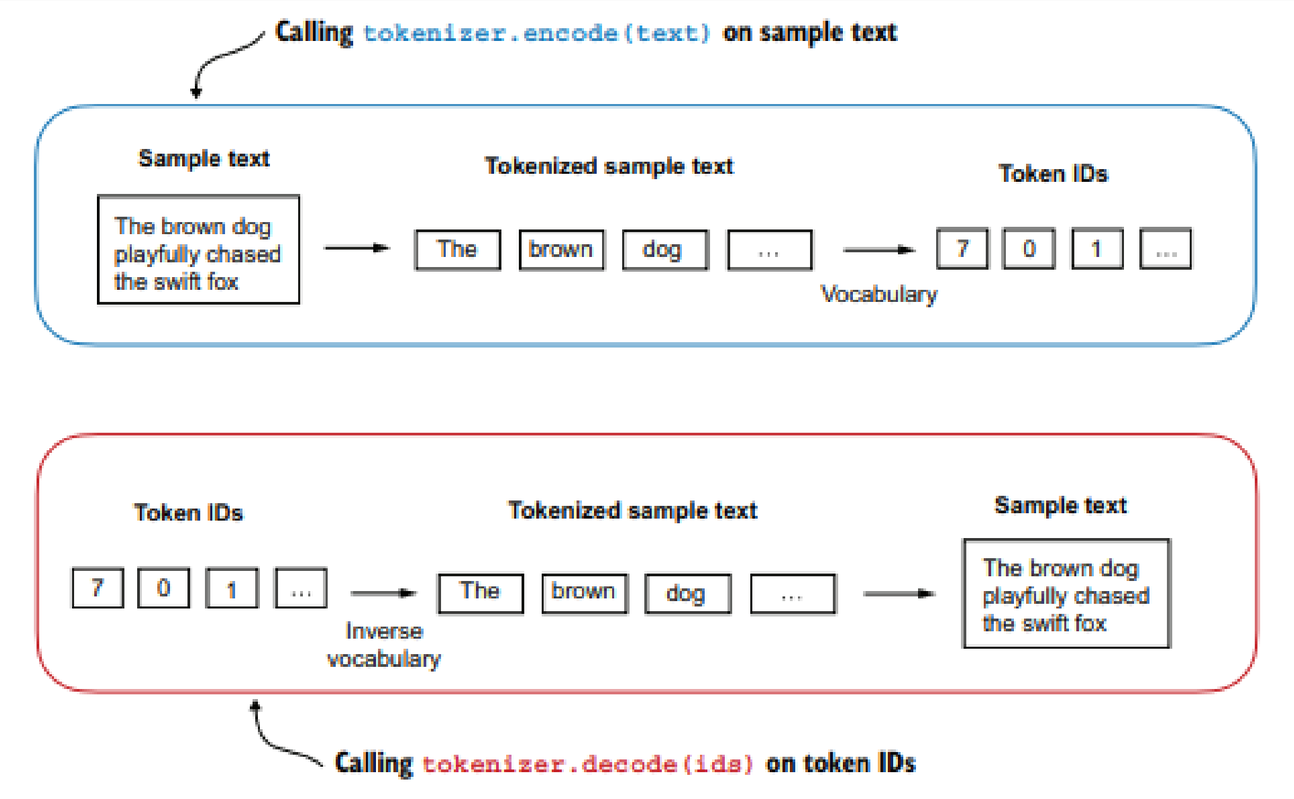


Figura 2.8. O método encode divide o texto em tokens e os converte em token IDs por meio do vocabulário. O método decode converte os token IDs de volta em tokens e os concatena em texto natural.

Special Context Tokens (Tokens Especiais)

Uma palavra fora do vocabulário gera erro no tokenizador simples. Por isso o vocabulário é estendido com tokens especiais: <|unk|>, para palavras desconhecidas, e <|endoftext|>, para separar textos.

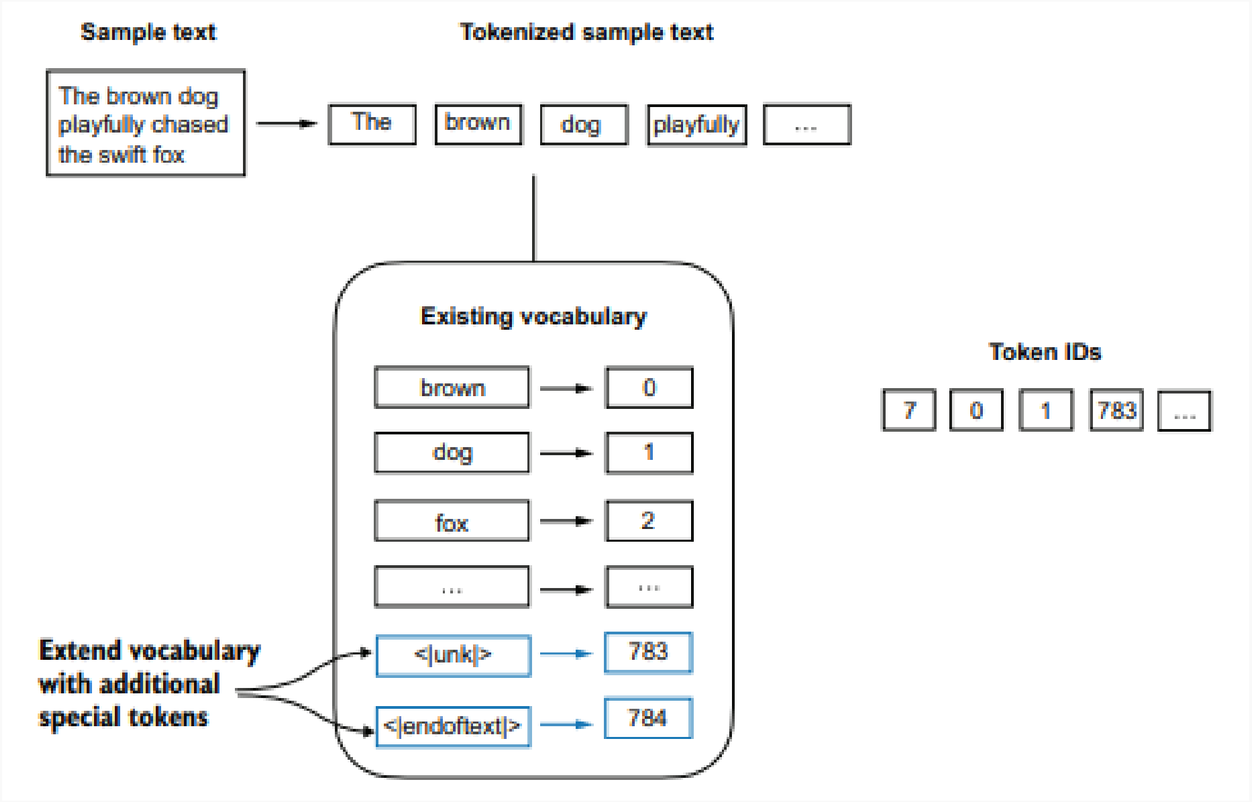


Figura 2.9. Tokens especiais são adicionados ao vocabulário: <|unk|> representa palavras desconhecidas e <|endoftext|> separa duas fontes de texto sem relação entre si. Ambos recebem os últimos token IDs.

<|endoftext|> entre textos independentes

No treinamento com vários documentos, o token <|endoftext|> é inserido entre eles para indicar que, embora concatenados, os textos não estão relacionados. O GPT usa apenas esse token.

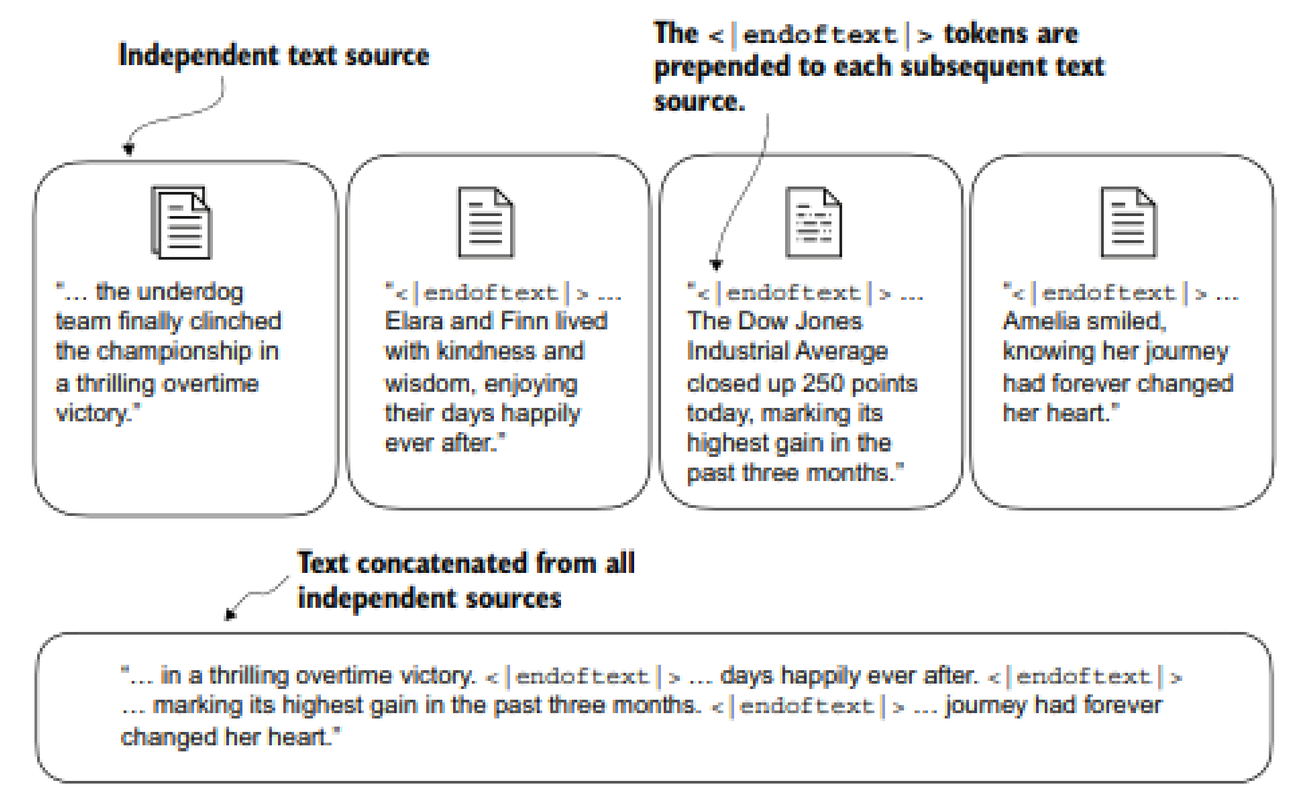


Figura 2.10. Textos independentes são concatenados com o token <|endoftext|> entre eles, que marca o início ou o fim de cada segmento e ajuda o modelo a processá-los de forma mais eficaz.

Byte Pair Encoding (BPE)

O BPE (Codificação por Pares de Bytes) divide palavras desconhecidas em subpalavras e caracteres, dispensando o token <|unk|>. É o tokenizador do GPT-2, do GPT-3 e do ChatGPT original, com vocabulário de 50.257 tokens.

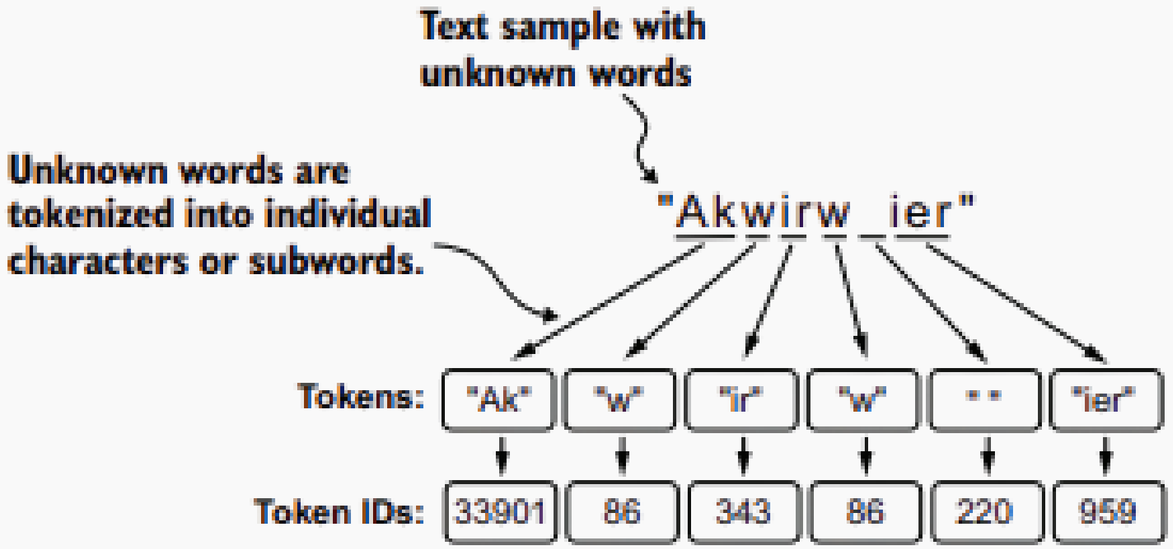


Figura 2.11. O BPE decompõe palavras desconhecidas em subpalavras e caracteres individuais. Assim, qualquer palavra pode ser processada sem ser substituída por um token especial.

Sliding Window (Janela Deslizante)

LLMs são pré-treinados prevendo o próximo token. A partir do texto tokenizado, criamos pares de entrada e alvo: o modelo vê os tokens anteriores e não tem acesso ao que vem depois do alvo.

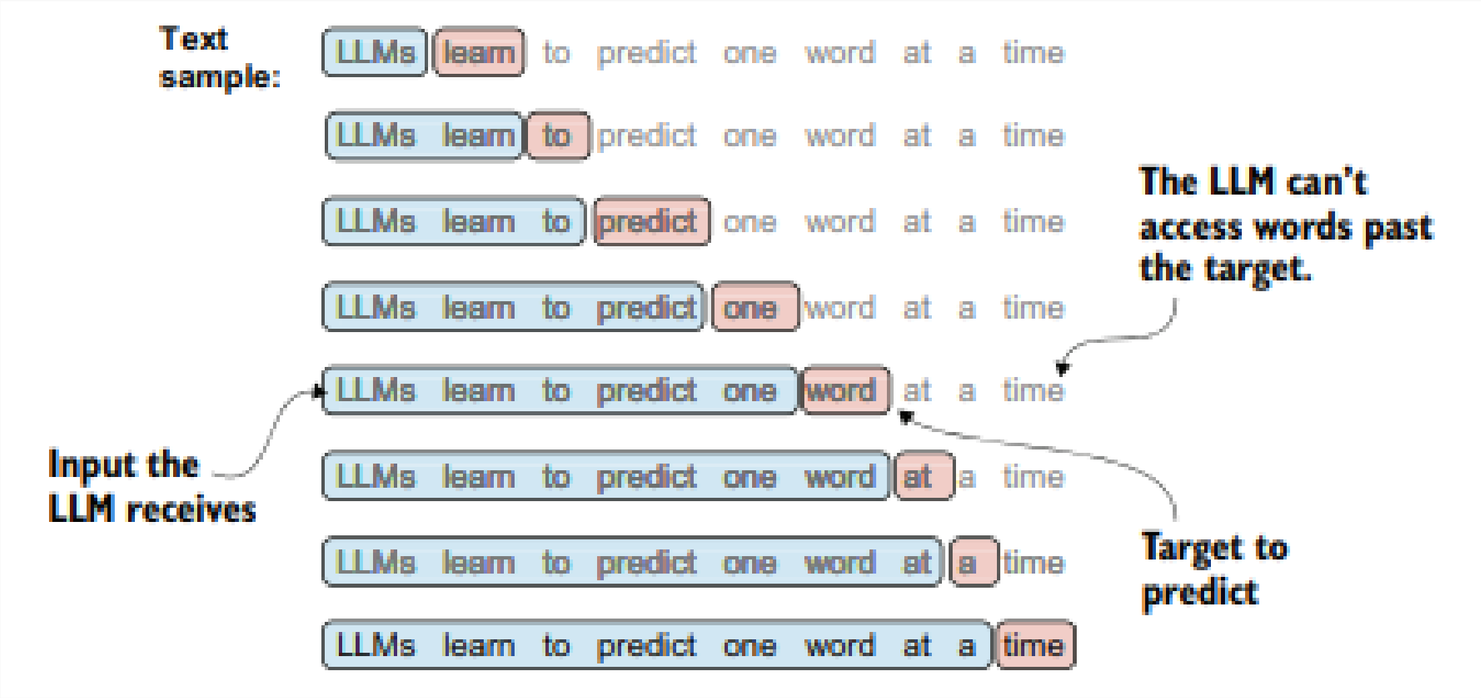


Figura 2.12. Blocos de entrada (em azul) são extraídos do texto, e o alvo (em vermelho) é a palavra seguinte ao bloco. Durante o treinamento, as palavras posteriores ao alvo ficam ocultas. A tokenização foi omitida para simplificar.

Entradas (x) e alvos (y)

Para o treinamento em lote, as entradas são organizadas em um tensor x e os alvos em um tensor y, que é x deslocado em uma posição. No PyTorch, isso é feito com as classes Dataset e DataLoader.

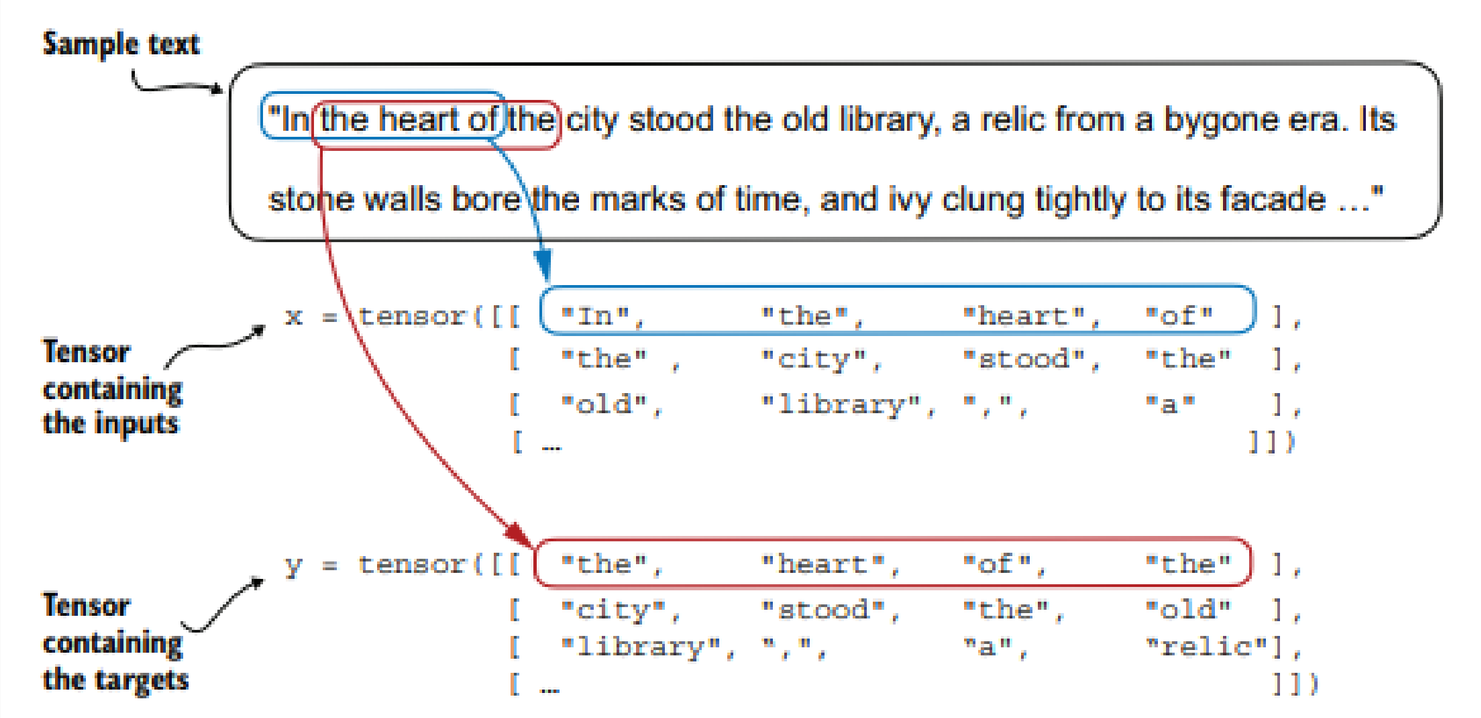


Figura 2.13. Cada linha de x é um contexto de entrada, e a linha correspondente de y contém os alvos, ou seja, as palavras seguintes obtidas ao deslocar a entrada em uma posição.

Stride: o passo da janela deslizante

O stride define quantas posições a janela avança entre um exemplo e o seguinte. Com stride igual ao tamanho da janela, não há sobreposição entre os lotes, o que reduz o risco de overfitting.

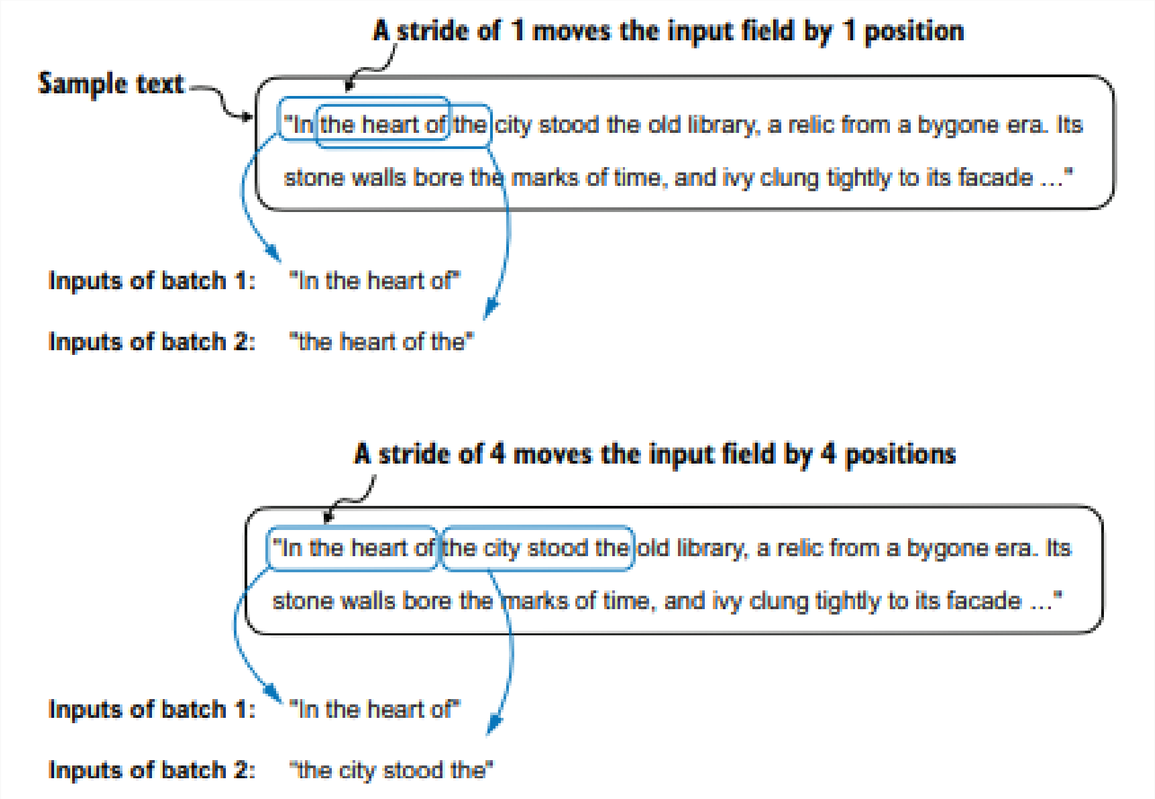


Figura 2.14. Com stride 1, a janela de entrada avança uma posição a cada lote; com stride 4, avança quatro posições e não há sobreposição entre os lotes.

Token Embeddings: a camada de embedding

O embedding layer é uma matriz de pesos, inicializada com valores aleatórios e otimizada durante o treinamento. Obter o embedding de um token ID é uma operação de consulta (lookup) a uma linha dessa matriz.

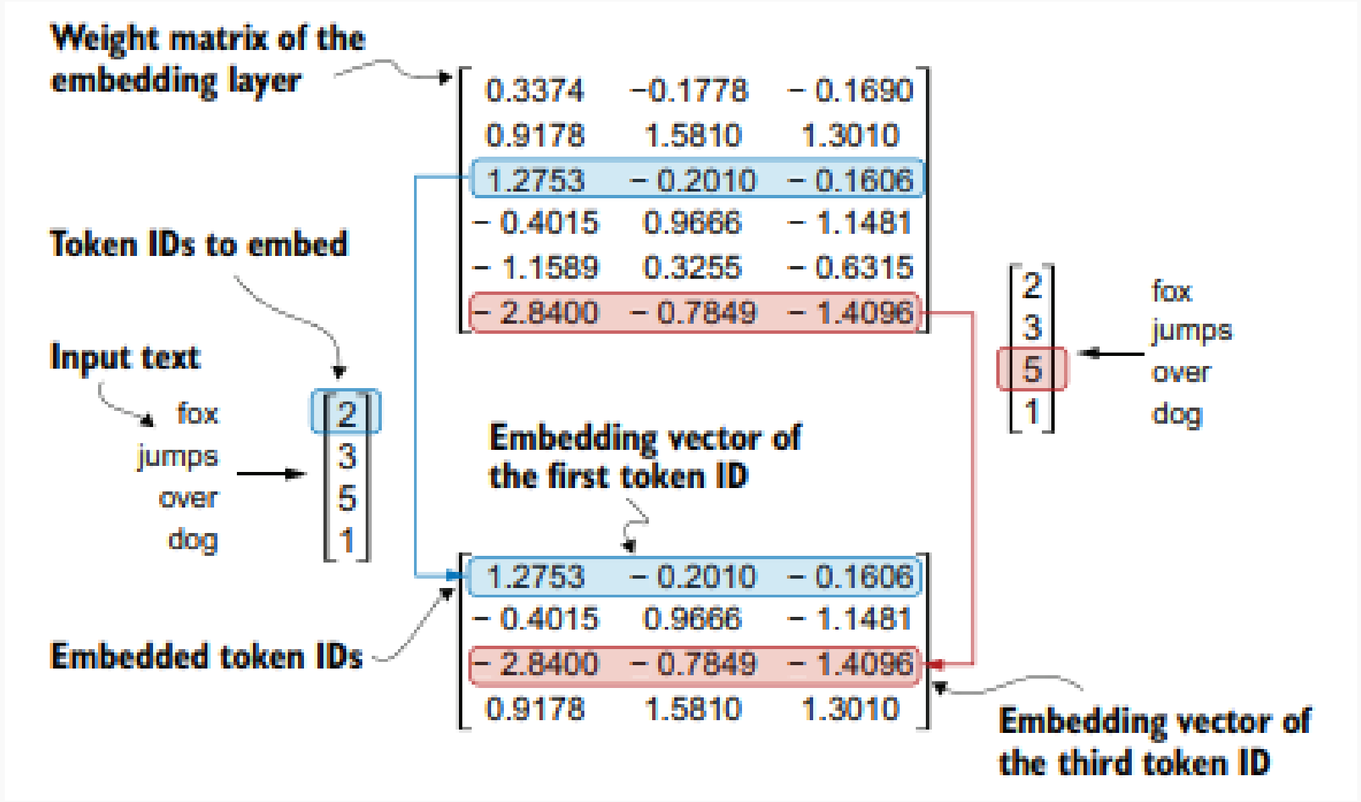


Figura 2.16. A camada de embedding recupera da matriz de pesos o vetor de cada token ID. O token ID 5, por exemplo, corresponde à sexta linha, pois a contagem em Python começa em 0.

O mesmo token ID gera sempre o mesmo vetor

A camada de embedding ignora a posição do token na sequência, e o mecanismo de self-attention também não tem noção de ordem. Por isso é necessário adicionar informação de posição.

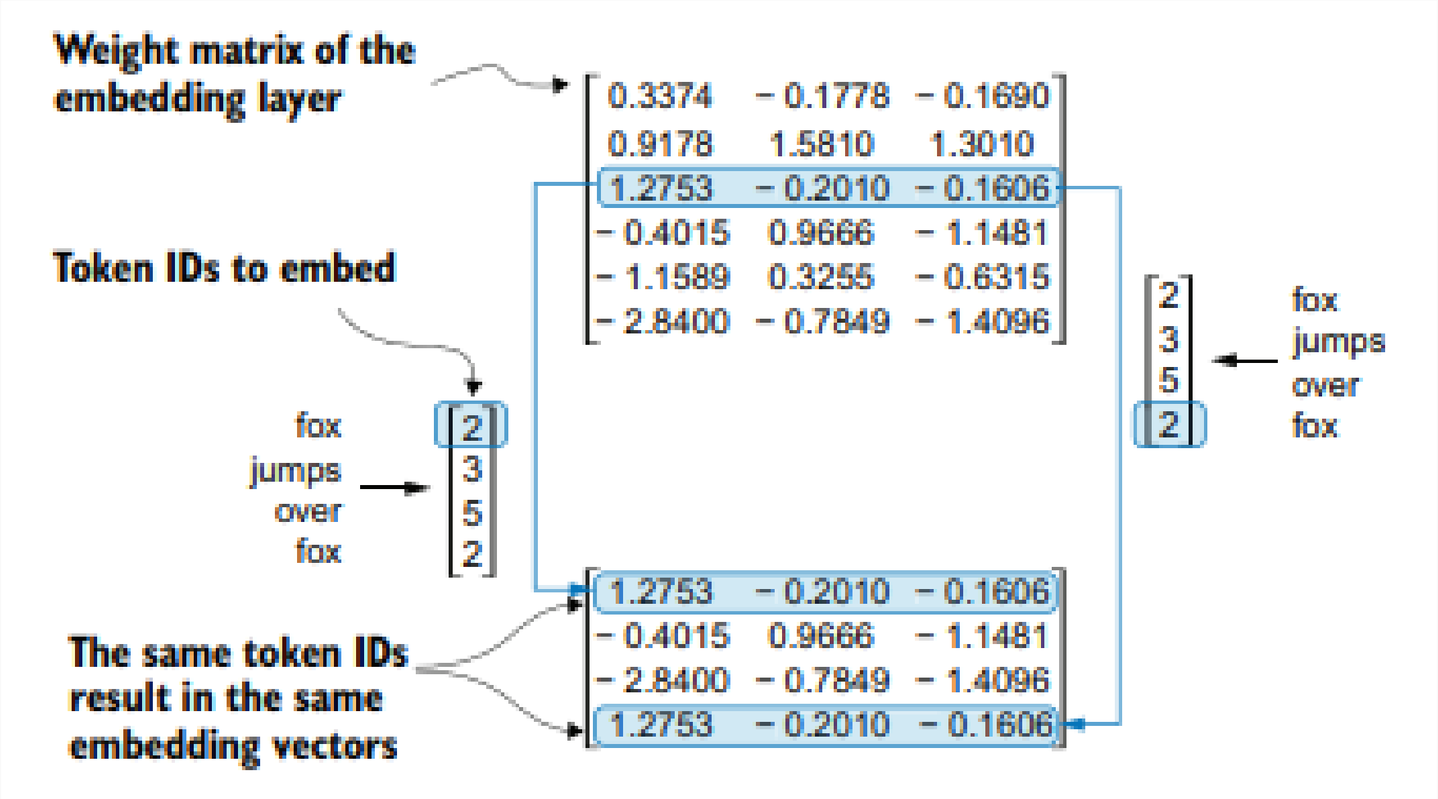


Figura 2.17. O mesmo token ID produz o mesmo vetor de embedding, independentemente de sua posição na sequência de entrada.

Positional Embeddings (Embeddings Posicionais)

Podem ser absolutos, associados a posições específicas da sequência, ou relativos, baseados na distância entre tokens.
Os modelos GPT usam embeddings posicionais absolutos, otimizados durante o treinamento.

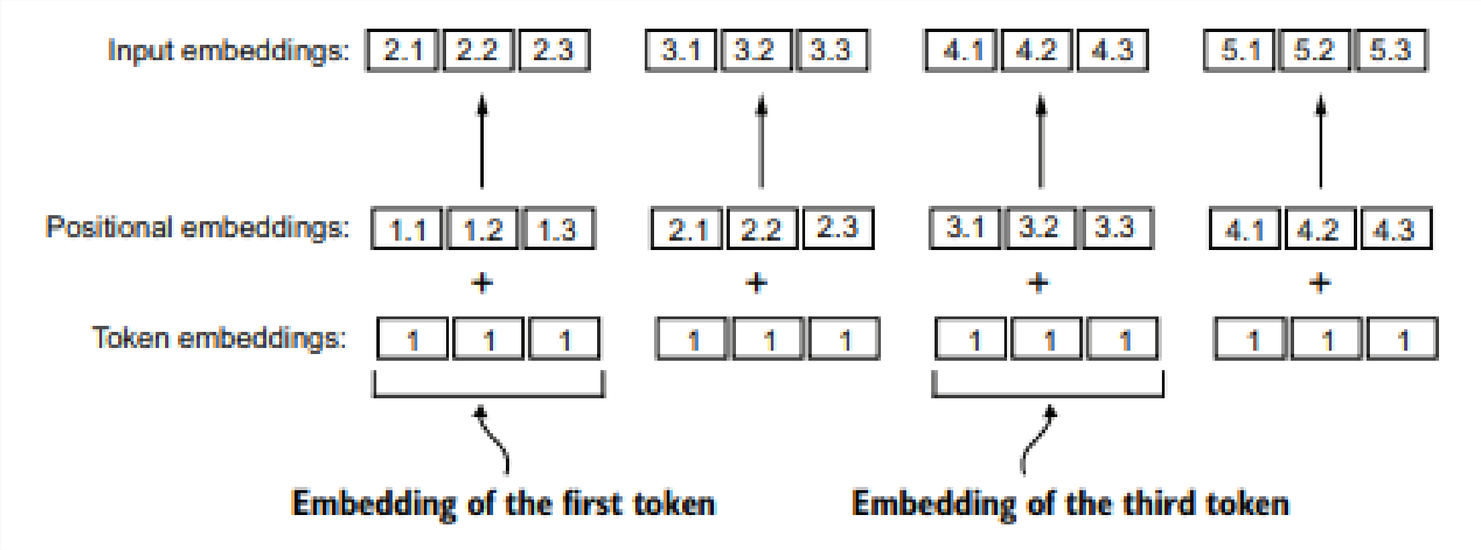


Figura 2.18. Os embeddings posicionais são somados aos embeddings de token para formar os input embeddings. Eles têm a mesma dimensão dos embeddings de token, que aqui aparecem com valor 1 por simplicidade.

Input Embeddings: o pipeline completo

O texto é dividido em tokens, convertido em token IDs, transformado em token embeddings e combinado com embeddings posicionais, gerando os input embeddings que alimentam as camadas principais do LLM.

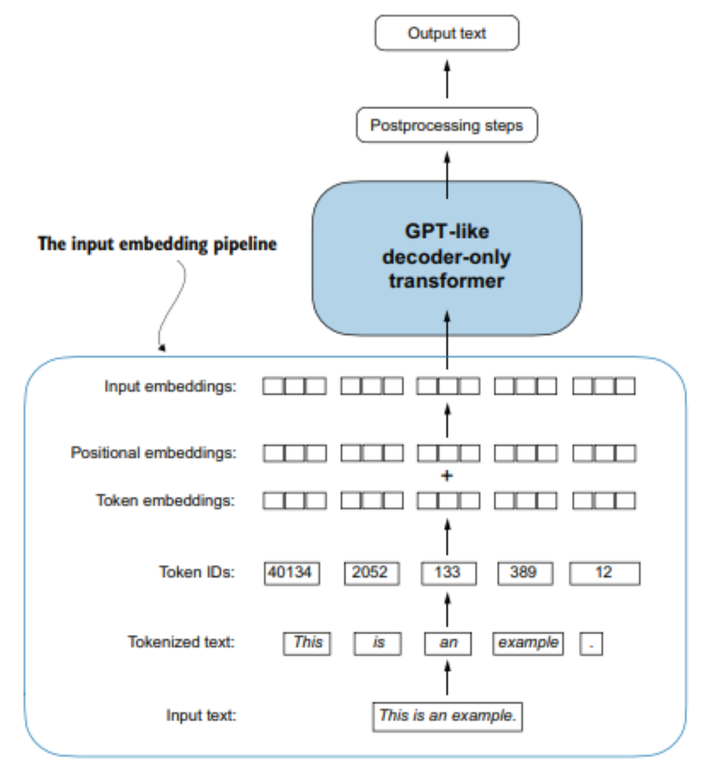


Figura 2.19. Pipeline de processamento da entrada: tokens, token IDs, token embeddings somados aos embeddings posicionais e, por fim, os input embeddings usados pelo LLM.

Capítulo 3

O problema das sequências longas

A motivação para attention: o cenário antes dos transformers

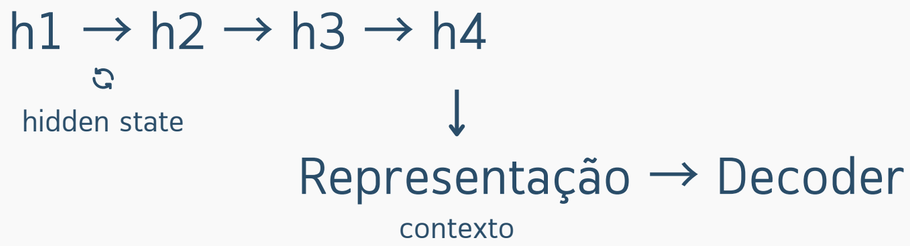
Encoder
Recebe a sequência de entrada e transforma em representação.

Decoder
Recebe a representação e produz a sequência de saída.

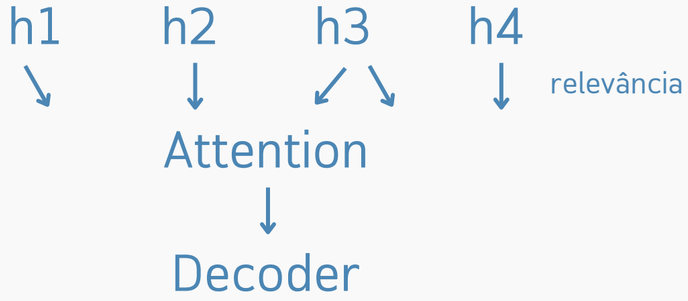
Onde a dificuldade aparece?
Em sequências curtas, essa compressão de contexto pode funcionar razoavelmente. Em sequências mais complexas, há uma dificuldade de preservar informações relevantes de partes distantes da sequência.

Sem e com Attention

O encoder precisa ir acumulando informações:



Agora o decoder pode consultar os diferentes estados do encoder:



Attention Weights

Para produzir um determinado token de saída, o modelo não precisa considerar todos os tokens da entrada com a mesma intensidade. Alguns elementos podem ser mais relevantes que os outros.

O score indica a compatibilidade/relevância entre a Query e uma Key. O Softmax transforma esses scores em pesos normalizados, permitindo interpretar a contribuição relativa de cada elemento.

Scores
h1 → 0.2
h2 → 0.3
h3 → 0.1
h4 → 0.9

Softmax
→

Weights
h1 → 0.10
h2 → 0.15
h3 → 0.05
h4 → 0.70

O modelo transforma os scores em valores que podem ser interpretados como pesos:
0.10 + 0.15 + 0.05 + 0.70 = 1.00

Q, K e V

Query, key e value

Query

O que estou procurando agora?

Pergunta

Key

Que tipo de informação eu tenho?

Identifica

Value

Que tipo de informação eu vou pegar?

Entrega

De onde surgem Q, K e V

Q = XWQ

K = XWK

V = XWV

- X = embeddings
- WQ = matriz aprendida para produzir Queries
- WK = matriz aprendida para produzir Keys
- WV = matriz aprendida para produzir Values

Essas matrizes WQ, WK e WV são parâmetros aprendidos durante o treinamento.

O que é dot product?

Dot product (produto escalar) é uma operação entre dois vetores que resulta em um único número.

Exemplo

Qgosto = [0.2, 0.5, 0.1]

*K*pizza = [0.4, 0.5, 0.3]

Qgosto · *K*pizza = (0.2×0.4) + (0.5×0.5) + (0.1×0.3)

= 0.08 + 0.25 + 0.03 = 0.36

Esse 0.36 é o **attention score** entre "gosto" e "pizza" nesse exemplo

Por que isso indica "compatibilidade"?

Pensa que a Query representa: "O que eu estou procurando?"

E a *K*ey representa: "Que tipo de informação eu tenho?"

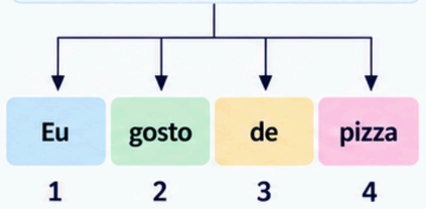
O dot product compara essas duas coisas, se os vetores forem orientados de maneira semelhante, o resultado tende a ser maior.

Attention scores

1. Frase e tokenização

A frase é dividida em tokens (palavras).

"Eu gosto de pizza"



2. Q, K e V

Cada token é convertido em três vetores usando matrizes aprendidas (W_Q, W_K, W_V).

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Token	Query (Q)	Key (K)	Value (V)
Eu	Q_1	K_1	V_1
gosto	Q_2	K_2	V_2
de	Q_3	K_3	V_3
pizza	Q_4	K_4	V_4

3. Escolhendo um token

Vamos calcular os attention scores para o token "gosto" (posição 2). Usamos a sua Query (Q_2) e comparamos com todas as Keys.

Query de "gosto"
Q_2

↓

Comparada com todas as Keys (K_1, K_2, K_3, K_4)

4. Produto escalar

O attention score é o produto escalar entre a Query e cada Key:

$$score_{2,j} = Q_2 \cdot K_j$$

Como funciona o produto escalar?

Se $Q_2 = [q_1, q_2, q_3]$ e $K_j = [k_1, k_2, k_3]$, então:

$$Q_2 \cdot K_j = q_1k_1 + q_2k_2 + q_3k_3$$

5. Exemplo numérico

Vamos usar valores fictícios para ilustrar o cálculo dos scores da palavra "gosto".

Suponha:

$Q_2 = [0.2, 0.5, 0.1]$

Keys de cada token:

K_1 (Eu) $= [0.1, 0.4, 0.2]$
K_2 (gosto) $= [0.3, 0.6, 0.1]$
K_3 (de) $= [0.0, 0.2, 0.4]$
K_4 (pizza) $= [0.4, 0.5, 0.3]$

6. Cálculo dos attention scores de "gosto"

Calculamos o produto escalar de Q_2 com cada Key.

Token (j)	Cálculo	Resultado ($score_{2,j}$)
Eu (1)	$0.2 \cdot 0.1 + 0.5 \cdot 0.4 + 0.1 \cdot 0.2$	**0.24**
gosto (2)	$0.2 \cdot 0.3 + 0.5 \cdot 0.6 + 0.1 \cdot 0.1$	**0.43**
de (3)	$0.2 \cdot 0.0 + 0.5 \cdot 0.2 + 0.1 \cdot 0.4$	**0.14**
pizza (4)	$0.2 \cdot 0.4 + 0.5 \cdot 0.5 + 0.1 \cdot 0.3$	**0.41**

7. Attention scores da palavra "gosto"

O vetor de attention scores contém a compatibilidade da Query de "gosto" com cada token da sequência.

Token	Attention Score ($Q_2 \cdot K_j$)
Eu	**0.24**
gosto	**0.43**
de	**0.14**
pizza	**0.41**

Esses são os attention scores.
Eles ainda não são os pesos de atenção (attention weights).

8. Próximo passo

Os attention scores passam por dois passos antes de se tornarem os attention weights:

Attention scores (QK^T)

↓

Divisão por $\sqrt{d_k}$

↓

Softmax

↓

Attention weights (distribuição de probabilidades)

Self-Attention

1. Frase e tokenização

A frase é dividida em tokens (palavras).

"Eu gosto de pizza"



2. Embeddings

Cada token é convertido em um vetor.

Token	Embedding (X)
Eu	[0.1 0.2 ⋯ 0.4]
gosto	[0.3 −0.1 ⋯ 0.5]
de	[0.0 0.6 ⋯ −0.2]
pizza	[−0.3 0.8 ⋯ 0.1]

3. Q, K e V

Cada embedding é projetado em três vetores usando matrizes aprendidas (W_Q, W_K, W_V).

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Token	Query (Q)	Key (K)	Value (V)
Eu	Q_1	K_1	V_1
gosto	Q_2	K_2	V_2
de	Q_3	K_3	V_3
pizza	Q_4	K_4	V_4

4. Attention scores

Para cada token, a Query é comparada com todas as Keys (produto escalar).

Exemplo: scores da palavra "gosto" (Q_2)

$Q_2 \cdot K_1$	$Q_2 \cdot K_2$	$Q_2 \cdot K_3$	$Q_2 \cdot K_4$
Eu	gosto	de	pizza
0.2	0.8	0.1	0.7

5. Escalamento e Softmax

Os scores são divididos por $\sqrt{d_k}$ e depois passam pelo Softmax, gerando os attention weights.

$$\alpha_{2,j} = \text{softmax}\left(\frac{Q_2 \cdot K_j}{\sqrt{d_k}}\right)$$

Attention weights da palavra "gosto"

Eu	gosto	de	pizza
0.12	0.30	0.08	0.50

Os pesos somam 1:
0.12 + 0.30 + 0.08 + 0.50 = 1.00

6. Combinação dos Values

Os attention weights são usados para fazer uma soma ponderada dos Values.

Novo vetor da palavra "gosto"

$$z_2 = 0.12 \cdot V_1 + 0.30 \cdot V_2 + 0.08 \cdot V_3 + 0.50 \cdot V_4$$

Eu	gosto	de	pizza
0.12	0.30	0.08	0.50



Novo vetor contextualizado de "gosto"

7. Resultado para todos os tokens

O mesmo processo é feito para todas as palavras da sequência.

Token	Nova representação
Eu	z_1 (novo vetor de "Eu")
gosto	z_2 (novo vetor de "gosto")
de	z_3 (novo vetor de "de")
pizza	z_4 (novo vetor de "pizza")

8. Self-Attention completo (forma matricial)

De forma matricial, fazemos isso para todos os tokens ao mesmo tempo.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

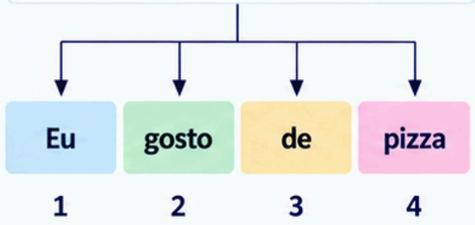
Q	Matriz de Queries	$(n_{tokens} \times d_k)$
K	Matriz de Keys	$(n_{tokens} \times d_k)$
V	Matriz de Values	$(n_{tokens} \times d_v)$
QK^T	Matriz de scores	$(n_{tokens} \times n_{tokens})$
softmax	Aplicado por linha (gerando os attention weights)	
Resultado	$(n_{tokens} \times d_v)$	

Context Vector

1. Frase e tokenização

A frase é dividida em tokens (palavras).

"Eu gosto de pizza"



2. Q, K e V

Cada token é convertido em três vetores usando matrizes aprendidas (W_Q, W_K, W_V).

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Token	Query (Q)	Key (K)	Value (V)
Eu	Q_1	K_1	V_1
gosto	Q_2	K_2	V_2
de	Q_3	K_3	V_3
pizza	Q_4	K_4	V_4

3. Attention weights de "gosto"

Calculamos os attention scores, aplicamos o escalonamento e o Softmax, obtendo os attention weights para o token "gosto" (Q_2).

Token	Attention weight ($\alpha_{2,j}$)
Eu	0.12
gosto	0.30
de	0.08
pizza	0.50

Esses pesos indicam o quanto "gosto" deve considerar cada palavra da sequência.

4. Valores (V)

Cada token possui um vetor de Value (V_j), que contém a informação a ser combinada.

Token	Value (V_j)
Eu	$V_1 = [1, 2]$
gosto	$V_2 = [3, 4]$
de	$V_3 = [5, 6]$
pizza	$V_4 = [7, 8]$

Os Values são os vetores que serão combinados usando os attention weights.

5. Cálculo do context vector

O context vector de "gosto" é a soma ponderada dos Values, usando os attention weights.

$$z_2 = \sum_{j=1}^{4} \alpha_{2,j} \cdot V_j$$

Substituindo os valores do exemplo:

$$z_2 = 0.12 \cdot V_1 + 0.30 \cdot V_2 + 0.08 \cdot V_3 + 0.50 \cdot V_4$$
$$= 0.12 \cdot [1, 2] + 0.30 \cdot [3, 4] + 0.08 \cdot [5, 6] + 0.50 \cdot [7, 8]$$

6. Cálculo passo a passo

Calculamos cada dimensão do vetor resultante.

Primeira dimensão:

$0.12 \cdot 1 + 0.30 \cdot 3 + 0.08 \cdot 5 + 0.50 \cdot 7$
$= 0.12 + 0.90 + 0.40 + 3.50 =$ **4.92**

Segunda dimensão:

$0.12 \cdot 2 + 0.30 \cdot 4 + 0.08 \cdot 6 + 0.50 \cdot 8$
$= 0.24 + 1.20 + 0.48 + 4.00 =$ **5.92**

7. Context vector de "gosto"

O resultado é um novo vetor que representa "gosto" considerando o contexto da frase.

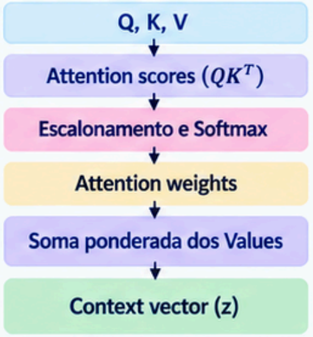
Context vector de "gosto"

$$z_2 = [4.92, 5.92]$$

Esse vetor contém informações de várias palavras da sequência, ponderadas pelos attention weights.

8. Visão geral do processo

O context vector é obtido após calcular os attention weights e combinar os Values.



Máscara casual

Impede que o modelo veja tokens futuros durante a geração.

	Eu	gosto	de	pizza
Eu	✓	✓	✓	✓
gosto	✓	✓	✓	✓
de	✓	✓	✓	✓
pizza	✓	✓	✓	✓

	Eu	gosto	de	pizza
Eu	✓	✗	✗	✗
gosto	✓	✓	✗	✗
de	✓	✓	✓	✗
pizza	✓	✓	✓	✓

Multi-head attention

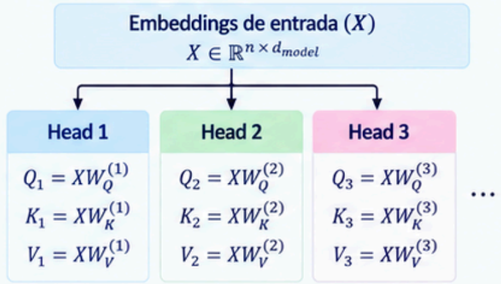
1. Ideia geral

O Multi-Head Attention executa vários Self-Attentions em paralelo (heads), cada um com suas próprias projeções de Q, K e V.



2. Projeções de Q, K e V por head

Cada head possui suas próprias matrizes de projeção, que transformam os embeddings de entrada em Q, K e V.



Cada head aprende projeções diferentes e pode capturar relações diferentes na sequência.

3. Attention em cada head

Cada head calcula o Self-Attention normalmente, com seus próprios Q, K e V.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Para o head i:

$$head_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right)V_i$$

O resultado de cada head é um conjunto de context vectors, um para cada token da sequência.

4. Exemplo intuitivo: diferentes relações

Cada head pode aprender a focar em relações diferentes da frase.



Esses são apenas exemplos intuitivos.
Na prática, os heads aprendem as relações mais úteis durante o treinamento.

5. Saídas dos heads

Cada head produz um conjunto de vetores contextualizados (um para cada token).

Head 1	Head 2	Head 3
$Z_1 = \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \\ z_3^{(1)} \\ z_4^{(1)} \end{bmatrix}$	$Z_2 = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \\ z_4^{(2)} \end{bmatrix}$	$Z_3 = \begin{bmatrix} z_1^{(3)} \\ z_2^{(3)} \\ z_3^{(3)} \\ z_4^{(3)} \end{bmatrix}$ ...
$\mathbb{R}^{n \times d_h}$	$\mathbb{R}^{n \times d_h}$	$\mathbb{R}^{n \times d_h}$

Todos os heads geram vetores com a mesma dimensão d_h (geralmente $d_h = d_{model}$ / número de heads).

6. Concatenação

Os resultados de todos os heads são concatenados ao longo da dimensão dos vetores.

Token	Head 1	Head 2	Head 3
Eu	$z_1^{(1)}$	$z_1^{(2)}$	$z_1^{(3)}$
gosto	$z_2^{(1)}$	$z_2^{(2)}$	$z_2^{(3)}$
de	$z_3^{(1)}$	$z_3^{(2)}$	$z_3^{(3)}$
pizza	$z_4^{(1)}$	$z_4^{(2)}$	$z_4^{(3)}$

$$\text{Concat}(Z_1, Z_2, Z_3) \in \mathbb{R}^{n \times (h \cdot d_h)}$$

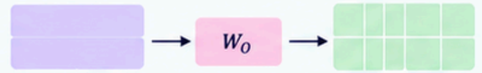
A concatenação junta as informações aprendidas por todos os heads.

7. Projeção de saída

O vetor concatenado passa por uma transformação linear usando a matriz W_O, gerando a saída final.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)\, W_O$$

$$W_O \in \mathbb{R}^{(h \cdot d_h) \times d_{model}}$$



$\mathbb{R}^{n \times (h \cdot d_h)}$ (vetores concatenados)

$\mathbb{R}^{n \times d_{model}}$ (saída do Multi-Head Attention)

A projeção de saída combina as informações dos vários heads e retorna um vetor com a mesma dimensão do modelo (d_{model}).

8. Resumo do processo

Passo a passo do Multi-Head Attention.

1. **Entradas:** embeddings da sequência (X).
2. Para cada head, calcular Q_i, K_i, V_i com matrizes próprias.
3. Cada head faz Self-Attention e produz seus vetores.
4. Concatenar os resultados de todos os heads.
5. Aplicar a projeção linear W_O.
6. Obter a saída final com a mesma dimensão do modelo.

O Multi-Head Attention permite que o modelo capture diferentes tipos de relações simultaneamente, tornando as representações mais ricas e contextualizadas.

O caminho de um token

Agradecimentos

Email da Equipe:

cmc@ic.ufal.br · eaf@ic.ufal.br · lfmn@ic.ufal.br · arjs@ic.ufal.br · vlll@ic.ufal.br